GU?A FUNCIONAL DEL SISTEMA

Funcionamiento y flujo CRUD

Qu? hace cada m?dulo y c?mo se implementan sus operaciones

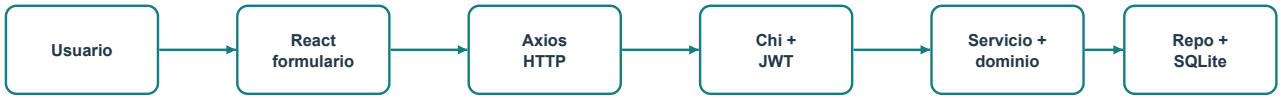
ADMISI?N	NOTAS	ADMINISTRACI?N
Patr?n State	Patr?n Memento	CRUD por capas

Gu?a basada en el c?digo actual. Distingue las funciones disponibles en la interfaz, las rutas existentes en el backend y las pantallas que todav?a no persisten informaci?n.

Incluye flujos, operaciones Create / Read / Update / Delete, m?todos de servicio y repositorio, y patrones de dise?o.

Esta guía complementa Arquitectura y funcionamiento: describe qué hace cada módulo, qué operaciones de crear, consultar, actualizar y eliminar están conectadas, y qué patrón o método organiza cada flujo. Se distingue entre una operación disponible en el backend y una acción que el usuario puede ejecutar actualmente en la interfaz.

Recorrido general de una operación



La respuesta JSON regresa por la misma cadena y actualiza la pantalla.

1. React recopila y valida la entrada visible y mantiene borradores locales.
2. Axios envía una solicitud HTTP al backend. Si hay sesión, añade el JWT.
3. Chi encuentra la ruta; el middleware autentica al usuario y verifica el rol.
4. El handler traduce HTTP a un DTO del servicio y convierte el resultado en estado HTTP y JSON.
5. El servicio aplica reglas de negocio y llama a una interfaz de repositorio.
6. El repositorio SQLite consulta o cambia las tablas. Algunas operaciones son transaccionales.
7. Axios entrega los datos a React, que refresca la vista o muestra un error.

`CRUD` significa Crear (`Create`), Leer (`Read`), Actualizar (`Update`) y Eliminar (`Delete`). Una pantalla puede tener solo una parte del CRUD: no se considera conectado algo que únicamente dibuja un formulario o una tabla.

Resumen de cobertura actual

Módulo	Crear	Leer	Actualizar	Eliminar	Estado real
Solicitud pública / admisiones	Sí, formulario público	Sí, listado y detalle para Secretaría/Dirección	Parcial: cambio de estado en la UI; edición general existe en API pero no en esta pantalla	API permite eliminar solicitudes no aprobadas; no se ofrece botón en la UI	Conectado, con operaciones administrativas de API no expuestas en la pantalla
Notas	Sí, primera nota por estudiante y asignatura	Sí, listado, detalle e historial	Sí, guardar nueva versión, deshacer, rehacer y restaurar	API tiene borrado de nota, pero no se ofrece en la UI	Conectado con Memento
Docentes / usuarios	Sí, crear cuentas desde API de Dirección	Sí, directorio y asignaciones	Solo activar/desactivar; no editar perfil desde UI/API disponible	No se ofrece eliminar usuario	CRUD parcial
Cursos / asignaturas	Sí, asignatura nueva desde API	Sí, catálogos	No hay actualización disponible	No hay eliminación disponible	Crear y consultar
Asignaciones docentes	Sí, asignar docente a curso, grado y sección	Sí, consultar asignaciones	No hay edición disponible	Sí, API permite eliminar asignación	Crear, consultar y eliminar
Asistencia	No	Lista de alumnos reutilizada desde notas/matrículas	Solo selección temporal en pantalla	No	Previsualización local; no se guarda
Clases / horarios	No se crean horarios	Se consultan asignaciones, no bloques de horario reales	No	No	Horario sin endpoints; interfaz informa la limitación
Dashboard	No aplica	Sí, indicadores y actividad disponibles según rol	No	No	Agrega lecturas existentes; indicadores no conectados se muestran como pendientes

1. Solicitud de matrícula y gestión de admisiones

Crear: formulario público

El asistente de la interfaz separa el ingreso en tres pasos: estudiante, apoderado y documento. Al avanzar, valida los campos requeridos del paso actual; al volver, conserva los datos. Al enviar el último paso, manda los datos y el archivo opcional a `POST /api/enrollments/public`.

El flujo del backend:

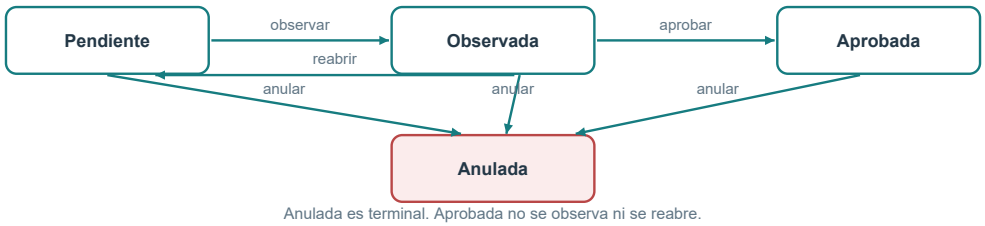
1. `EnrollmentHandler.RegisterPublic` lee el formulario y acepta un documento PDF, PNG o JPEG.
2. `EnrollmentService.RegisterPublic` valida los datos obligatorios, crea objetos `Student`, `Guardian` y `Enrollment`, genera código `SOL` y asigna el estado inicial Pendiente.
3. `EnrollmentRepository.Create` inicia una transacción y guarda estudiante, apoderado y solicitud.
4. Si todo se guarda, confirma la transacción y devuelve el expediente. Si una inserción falla, revierte la operación completa.

Métodos/patrones: DTO (`PublicEnrollmentDTO`), servicio de aplicación (`RegisterPublic`), repositorio (`Create`) y transacción SQLite. El formulario público no requiere sesión.

Leer: revisión y detalle

Secretaría y Dirección consultan lista, filtros de estado/búsqueda, detalle, documento adjunto y estadísticas. `EnrollmentService.List` y `GetByID` delegan al repositorio, que reúne datos relacionados de estudiante y apoderado. El filtro visual adicional de la tabla se realiza en React.

Actualizar: estado con State



Anulada es terminal. Aprobada no se observa ni se reabre.

Desde el expediente se solicita confirmar, observar o anular y se registra el motivo. El frontend deshabilita acciones incompatibles con el estado. El handler ejecuta `PATCH /api/enrollments/{id}/state`; `TransitionStatus` obtiene el expediente y delega al objeto de estado a través de `Enrollment.TransitionTo`.

El patrón State cambia el comportamiento según el estado actual:

Estado actual	Operaciones permitidas
Pendiente	Aprobar, observar o anular
Observada	Aprobar, actualizar motivo de observación, reabrir como Pendiente o anular
Aprobada	Anular
Anulada	Ninguna; estado terminal

Al aprobar, el servicio cambia un código `SOL` por uno `MAT`. La entidad actualiza observación y fecha. Transiciones no permitidas regresan error de negocio y no se persisten.

La acción visual Retirar actualmente se envía como estado `Anulada`; el dominio no tiene estado `Retirada`. Finalizar está deshabilitado porque el API no implementa esa transición.

Otras operaciones disponibles solo en API

- `PUT /api/enrollments/{id}` actualiza grado, sección u observación mediante `EnrollmentService.Update`.
- `POST /api/enrollments/{id}/payment` registra o reemplaza el pago asociado. Si falta monto o comprobante se aplican valores predeterminados.

- `DELETE /api/enrollments/{id}` elimina solicitudes no aprobadas. El servicio rechaza borrar una solicitud aprobada para mantener auditoría.

La pantalla actual no presenta formulario de edición, botón de pago ni botón de eliminación. El endpoint de borrado tampoco es una transición del patrón State; es una eliminación física para expedientes no aprobados.

2. Notas e historial

Leer y filtrar

El módulo consulta `/api/grades`; el backend obtiene el ID docente desde el JWT y restringe los registros a ese docente. Se aplican filtros opcionales de grado y sección. La pantalla permite filtrar por sección, curso y periodo B1-B4 y muestra promedio y versión.

Crear o actualizar una nota



La interfaz edita un bimestre a la vez como borrador y envía el conjunto B1-B4 con `POST /api/grades`. El servicio `GradeService.SaveGrade` valida que cada nota esté entre 0 y 20 y busca la combinación de estudiante y asignatura:

- Si no existe, crea la calificación en versión 1, calcula el promedio y genera el primer Memento.
- Si existe, actualiza el conjunto de notas, incrementa la versión, recalcula el promedio y crea otro Memento.

El patrón Memento se organiza con tres responsabilidades:

1. Originator: `Grade` contiene el estado actual, calcula promedio con `CalculateAverage`, captura una instantánea con `CreateMemento` y recupera una con `RestoreMemento`.
2. Memento: `GradeMemento` conserva las notas, promedio, versión, autor, fecha y acción.
3. Caretaker: `MementoService` consulta y administra el historial sin modificar manualmente los datos de las instantáneas.

`CreateWithMemento` y `UpdateWithMemento` guardan nota e historial en una transacción. Así se evita una nota modificada sin su correspondiente versión.

Deshacer, rehacer y restaurar

- Deshacer: busca la instantánea de versión anterior y restaura la calificación.
- Rehacer: busca la versión siguiente, si todavía existe.
- Historial: muestra versiones, fecha, acción y los valores B1-B4.
- Restaurar: selecciona una instantánea concreta y valida que pertenezca a la nota elegida.
- Nueva edición después de deshacer: `UpdateWithMemento` elimina instantáneas de versiones futuras antes de guardar la nueva rama.

Aunque el repositorio contiene un método para eliminar una nota, las rutas de notas no publican un endpoint `DELETE` y la pantalla no tiene acción de borrado. Por tanto, el CRUD visible de notas no incluye Delete.

3. Docentes, usuarios y administración académica

Usuarios y docentes

Dirección lee cuentas mediante `GET /api/director/users` y puede crear una mediante `POST /api/director/users`. `DirectorService.CreateUser` valida campos esenciales, comprueba el nombre de usuario y genera el hash de contraseña con bcrypt antes de llamar a `UserRepository.Create`. La contraseña en texto no se devuelve como parte de la entidad serializada.

`PATCH /api/director/users/{id}/toggle` cambia el campo activo de una cuenta. Es una actualización de estado, no una operación para modificar nombre, correo, rol o contraseña. No hay endpoint para borrar usuarios. El directorio docente combina la lista de cuentas de rol docente y las asignaciones; para roles no directivos muestra docentes que aparecen en asignaciones.

Patrón usado: handler → servicio → repositorio, DTO de entrada, hashing bcrypt y autorización por rol. No hay un State de dominio independiente para las cuentas.

Asignaturas

El API permite consultar la lista y crear asignaturas con nombre, código y descripción. El repositorio SQLite inserta la asignatura. No hay endpoint de edición ni eliminación. La pantalla docente muestra asignaciones y cursos relacionados, pero no es una pantalla de administración CRUD completa.

Asignaciones docentes

Dirección crea la relación docente–asignatura–grado–sección y consulta asignaciones. `AssignTeacher` valida que todos los identificadores y referencias de texto estén presentes antes de invocar `CreateAssignment`. También puede eliminar una asignación con `DeleteAssignment`. No existe una ruta de actualización; para cambiar una asignación habría que eliminarla y crear otra.

Patrón usado: CRUD parcial mediante métodos explícitos de servicio y repositorio; no utiliza State ni Memento.

4. Asistencia y horarios

Asistencia

La pantalla obtiene una lista a partir de `/api/grades` para Docencia o `/api/enrollments` para otros roles, deduplica estudiantes y prepara opciones Presente, Tardanza, Falta y Justificada. Al seleccionar una opción solo actualiza estado React local. No hay ruta para consultar asistencias ni para guardarlas; el botón permanece deshabilitado y salir de la vista descarta la previsualización.

La tabla `asistencias` existe en las migraciones, pero no significa que exista un CRUD funcional: faltan handler, servicio, contrato de repositorio, endpoints y conexión de guardado desde el formulario.

Clases y horarios

La pantalla lee `/api/academic/assignments` y filtra relaciones existentes por sección o docente. La cuadrícula semanal es una estructura de presentación; no hay horas/aulas guardadas que la alimenten. Aunque la tabla `horarios` está preparada en la base, falta implementar el CRUD de horario de punta a punta.

5. Dashboard

El panel no modifica registros. Secretaría y Dirección combinan listado y estadísticas de solicitudes; Docencia y Dirección consultan notas según disponibilidad del API. La actividad reciente se arma ordenando y mostrando solicitudes o calificaciones ya recibidas.

Los KPIs de asistencia del día y clases programadas aparecen como – con una nota de que todavía no están conectados. No son resultados persistidos ni conteos calculados.

Métodos y patrones de diseño

Método o patrón	Uso real en el proyecto
CRUD por capas	Handlers traducen HTTP, servicios aplican reglas y repositorios implementan persistencia
DTO	Separa y valida datos que entran a los casos de uso
State	`Enrollment.TransitionTo` delega aprobar, observar, reabrir y anular al estado concreto
Memento	`Grade.CreateMemento` y `RestoreMemento` capturan/restauran versiones de notas
Repository	Interfaces separan servicios de SQL; SQLite aporta los adaptadores
Transacción SQLite	Agrupa creación de estudiante/apoderado/solicitud y escritura de nota/Memento
JWT + Role middleware	Protege rutas del backend por identidad y rol

Método o patrón	Uso real en el proyecto
Borrador local React	Permite editar formularios y notas antes de confirmar el guardado

En el flujo de admisión, las entidades relacionadas pueden verse como un agregado de creación transaccional, pero no hay una clase formal `EnrollmentAggregate`. Tampoco hay CRUD completo de horarios o asistencias todavía. La tabla académica nueva incluye estructuras adicionales; su existencia no implica que todos sus casos de uso ya estén conectados.

Ejecución para probar los flujos

En terminales separadas, desde la raíz del repositorio:

```
cd backend
go run ./cmd/api
```

```
cd frontend
npm ci
npm run dev
```

La API está en `http://localhost:8080` y la interfaz en `http://localhost:5173`. Las cuentas de demostración se documentan en la sección de credenciales del README.

Referencias

- Arquitectura y conexiones del sistema
- Diagrama de flujos de negocio
- Contrato HTTP implementado